FROZEN

performance-svc

EduPath :: AI Performance Intelligence Engine

16/16 Tests Passing · ERS Formula · Dropout Risk · TimescaleDB

PLATFORM	STACK	ARCHITECTURE	GENERATED
EduPath v1.0	Java 17 · Spring Boot 3.3.5	Hexagonal · Kafka · PostgreSQL	March 07, 2026

BUILD STATUS: SUCCESS · ALL TESTS PASSING · RECORD FROZEN

performance-svc — COMPLETION RECORD

STATUS: FROZEN — DO NOT MODIFY
Tests passed: 16/16 | Build: SUCCESS | Date: 2026-03-07

Service Overview

EduPath Student Career Portal — AI-powered performance intelligence engine. Computes the ExamReadiness Score (ERS) using a weighted multi-factor formula, tracks subject mastery levels, identifies weak areas, detects dropout risk via an LSTM-inspired heuristic, and maintains time-series performance snapshots for trend analysis. Consumes Kafka events from exam-tracker-svc and student-profile-svc.

Test Results

```
Tests run: 16, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS — Total time: 7.583 s
```

TEST CLASS	TESTS	RESULT
ArchitectureRulesTest	5	PASS
ReadinessScoreServiceTest	5	PASS
WeakAreaServiceTest	3	PASS
DropoutRiskCalculatorTest	3	PASS

Architecture

Standard hexagonal (ports & adapters). `domain` has zero Spring dependencies. Domain service `DropoutRiskCalculator` is a pure Java calculation engine.

```
com.edutech.performance/
  PerformanceSvcApplication.java
  api/
    ReadinessScoreController.java        REST — /api/v1/performance/readiness/**
    SubjectMasteryController.java        REST — /api/v1/performance/mastery/**
    WeakAreaController.java              REST — /api/v1/performance/weak-areas/**
    PerformanceDashboardController.java  REST — /api/v1/performance/dashboard/**
    GlobalExceptionHandler.java          RFC 7807 ProblemDetail
  application/
    service/
      ReadinessScoreService.java         ComputeReadinessScoreUseCase, GetReadinessScoreUseCase
      SubjectMasteryService.java         UpdateSubjectMasteryUseCase, GetSubjectMasteryUseCase
      WeakAreaService.java               RecordWeakAreaUseCase, GetWeakAreasUseCase
      PerformanceDashboardService.java   GetPerformanceDashboardUseCase
    dto/                                 Java records (all immutable)
    exception/                           ReadinessScoreNotFoundException, WeakAreaNotFoundException
  domain/
    model/
      ReadinessScore.java                @Entity, static factory compute(), @Version, soft delete
      SubjectMastery.java                @Entity, MasteryLevel enum
      WeakAreaRecord.java                @Entity, ErrorType classification
      PerformanceSnapshot.java           @Entity, time-series point (TimescaleDB hypertable)
    model/enums/                         MasteryLevel (BEGINNER, DEVELOPING, PROFICIENT, MASTERED)
                                         RiskLevel (GREEN, YELLOW, ORANGE, RED)
                                         ErrorType (CONCEPTUAL, COMPUTATIONAL, TIME, CARELESS)
    event/                               ReadinessScoreUpdatedEvent, WeakAreaDetectedEvent, Dropout
RiskEscalatedEvent
    service/
      DropoutRiskCalculator.java         Pure domain service — LSTM-inspired risk scoring
    port/in/                             7 use-case interfaces
    port/out/                            ReadinessScoreRepository, SubjectMasteryRepository, WeakAr
eaRepository,
```

```
                                        PerformanceSnapshotRepository, PerformanceEventPublisher
    infrastructure/
      persistence/                      SpringData*Repository (package-private) + public *Persiste
  nceAdapter @Component
      kafka/
        PerformanceKafkaAdapter.java    KafkaTemplate publisher, best-effort
        ExamEventConsumer.java          @KafkaListener — consumes exam.tracker.mock.completed
        StudentEventConsumer.java       @KafkaListener — consumes student.profile.created
      security/SecurityConfig.java       Trusts X-User-Id / X-User-Role headers (no JWT re-validat
  ion)
      config/
        KafkaTopicProperties.java        @ConfigurationProperties("kafka.topics")
        KafkaConsumerProperties.java     @ConfigurationProperties("spring.kafka.consumer")
```

ERS Formula

```
ERS = 0.25 × syllabusCoverage
    + 0.30 × mockTestTrend
    + 0.25 × masteryAverage
    + 0.10 × timeManagement
    + 0.10 × accuracyConsistency
```

Syllabus coverage: percentage of subjects with `MasteryLevel >= PROFICIENT` (ordinal > DEVELOPING). Topics at DEVELOPING or BEGINNER do not count as "covered".

MasteryLevel thresholds:
- `BEGINNER`: masteryPercent <= 30
- `DEVELOPING`: 31–60
- `PROFICIENT`: 61–80
- `MASTERED`: > 80

Flyway Migrations

VERSION	SCRIPT	DESCRIPTION
V1	V1__create_performance_schema.sql	CREATE SCHEMA performance_schema
V2	V2__create_readiness_scores.sql	readiness_scores table
V3	V3__create_subject_mastery.sql	subject_mastery table
V4	V4__create_weak_areas.sql	weak_area_records table
V5	V5__create_performance_snapshots.sql	performance_snapshots (TimescaleDB hypertable)

API Endpoints

METHOD	PATH	USE CASE
POST	`/api/v1/performance/readiness/{studentId}/{enrollmentId}`	Compute ERS
GET	`/api/v1/performance/readiness/{studentId}/{enrollmentId}/latest`	Get latest ERS
GET	`/api/v1/performance/readiness/{studentId}/{enrollmentId}/history`	ERS history
PATCH	`/api/v1/performance/mastery/{studentId}/{enrollmentId}/{subject}`	Update subject mastery
GET	`/api/v1/performance/mastery/{studentId}/{enrollmentId}`	Get all subject mastery
POST	`/api/v1/performance/weak-areas`	Record weak area
GET	`/api/v1/performance/weak-areas/{studentId}/{enrollmentId}`	Get weak areas
GET	`/api/v1/performance/dashboard/{studentId}/{enrollmentId}`	Full performance dashboard

Kafka Integration

Consumes:

TOPIC	CONSUMER	ACTION
`exam.tracker.mock.completed`	ExamEventConsumer	Updates mockTestTrendScore, triggers ERS recompute
`student.profile.created`	StudentEventConsumer	Initialises subject mastery records for new student

Publishes:

EVENT	TOPIC	TRIGGER
ReadinessScoreUpdatedEvent	`performance.readiness.updated`	ERS computed/updated
WeakAreaDetectedEvent	`performance.weak.area.detected`	Weak area recorded
DropoutRiskEscalatedEvent	`performance.dropout.risk`	Risk level RED detected

Key Design Decisions

- **ERS syllabus coverage threshold**: Only `PROFICIENT` + subjects count as covered (ordinal > DEVELOPING). Verified by `ReadinessScoreServiceTest.computeScore_success`.

- **DropoutRiskCalculator**: Pure domain service (no Spring), heuristic scoring on ERS trend, missed sessions, and weak area density. Risk levels: GREEN < 30, YELLOW < 50, ORANGE < 70, RED >= 70.
- **TimescaleDB**: `performance_snapshots` declared as hypertable for efficient time-series querying.
- **Kafka consumers**: `ExamEventConsumer` and `StudentEventConsumer` in infrastructure layer — Kafka dependency never leaks to domain.
- **Security**: Trusts `X-User-Id` / `X-User-Role` headers from student-gateway. No JWT re-validation.
- **No Lombok**: Manual constructors. Constructor injection only.

Fixes Applied

1. **ERS syllabus coverage**: Changed filter from `!= BEGINNER` to `ordinal() > MasteryLevel.DEVELOPING.ordinal()` — only PROFICIENT+ counts as covered. This correctly produces `37.50` for the test scenario (1 PROFICIENT + 1 DEVELOPING out of 2 subjects = 50% coverage).

ArchUnit Rules (5)

1. `domain` package has no dependencies on Spring Framework
2. `application` services depend on `domain` ports only (no infrastructure)
3. `@Entity` classes reside in `domain.model` package
4. `@Service` classes reside in `application.service` package
5. `infrastructure` adapters implement domain port interfaces

This document is permanently frozen. Any future changes to performance-svc require a new versioned record.